



Republic of Tunisia
*** * ***
Ministry of Youth and Sports
*** * ***
Discovery
Club Junior



ENGINEERING JOURNAL

DISCOVERY CLUB JUNIOR ROBOCUP GERMANY 2024

Topic:

RCJ Line Follower Robot

"The Line Follower"

"LAHBIB"

Prepared by:

Wissal Naouaii

Moemen Ayari

Ismail Mseddi

Koussay Othmani

Elaa Triki

Islem Aouadi

Year: 2023/2024

Contents

1	Presentation	1
1.1	Introduction	1
1.2	Problematic	1
1.2.1	Autonomous Navigation in Hazardous Environments	1
1.2.2	Accurate Victim Identification	2
1.2.3	Safe and Efficient Victim Transportation	2
1.2.4	Sustained Operation	2
1.3	Project Overview	2
2	Objectives	3
2.1	Autonomous Navigation	3
2.2	Detailed Description:	3
2.2.1	Terrain Adaptability:	3
2.2.2	Obstacle Avoidance:	3
2.2.3	Line Following:	3
2.2.4	Victim Identification	4
2.3	Detailed Description:	4
2.3.1	Sensor Integration:	4
2.3.2	Algorithm Development:	4
2.3.3	Environmental Adaptability:	4
3	Hardware Components	5
3.1	Microcontroller	5
3.2	Motors	5
3.3	Sensors	5
3.4	Actuators	6
3.5	Power Supply	7
4	Software Components	8
4.0.1	Programming Language	8
4.0.2	Libraries	8
4.0.3	Snippets of Code with Explanation	9
5	System Design	10
5.1	Mechanical Design	10
5.2	Electrical Design	10
5.3	Software Design	11

6	Software Architecture for Raspberry Pi-Based Autonomous Rescue Robot	13
6.1	Overview	13
6.2	Components of the Software Architecture	13
6.2.1	Operating System Layer	13
6.2.2	Middleware Layer	13
6.2.3	Application Layer	14
6.2.4	Communication Layer	14
6.3	Key Considerations	15
6.3.1	Real-Time Responsiveness	15
6.3.2	Fault Tolerance	15
6.3.3	Modularity and Extensibility	15
6.4	Conclusion	15
7	Performance and Testing for Raspberry Pi-Based Autonomous Rescue Robot	16
7.1	Overview	16
7.2	Performance Evaluation	16
7.2.1	Navigation Accuracy	16
7.2.2	Victim Identification and Transportation	17
7.2.3	Real-Time Responsiveness	17
7.3	Testing Procedures	17
7.3.1	Simulated Disaster Scenarios	17
7.3.2	Field Testing	18
7.4	Conclusion	18
8	Conclusion	19
9	Future Improvements	20

List of Figures

Chapter 1

Presentation

1.1 Introduction

In the realm of disaster response, the efficiency and safety of rescue operations are paramount. The use of autonomous systems in these operations can significantly enhance the speed and effectiveness of victim recovery while minimizing risks to human rescuers. Our project focuses on the development of an advanced autonomous rescue robot, designed to navigate complex and hazardous environments, identify and transport victims, and operate continuously until all victims are secured. This journal documents the systematic approach taken by our team to design, build, and test this robot, ensuring its capability to perform under challenging conditions.

1.2 Problematic

Natural disasters and catastrophic events often create environments that are unsafe for human rescuers, with obstacles such as debris, unstable structures, and hazardous materials. Traditional rescue operations in such conditions pose significant risks, and the need for rapid victim recovery is critical. This project addresses the following key problems:

1.2.1 Autonomous Navigation in Hazardous Environments

Developing a robot that can independently traverse uneven and obstructed terrain without human intervention.

1.2.2 Accurate Victim Identification

Ensuring the robot can reliably distinguish between live and deceased victims using sensory inputs.

1.2.3 Safe and Efficient Victim Transportation

Designing mechanisms that can securely and gently transport victims to designated safe zones.

1.2.4 Sustained Operation

Creating a system that can continue its mission autonomously over extended periods, maximizing the chances of successful rescues.

By addressing these challenges, our autonomous rescue robot aims to enhance the safety, efficiency, and effectiveness of disaster response efforts, ultimately saving lives and reducing the burden on human rescuers.

1.3 Project Overview

Our team is developing an autonomous rescue robot capable of navigating treacherous terrain and performing rescue missions without human assistance. The robot must follow a black line, navigate obstacles, and transport victims to a designated evacuation point. The robot's design focuses on durability, intelligence, and precision in navigation and handling of victims.

Chapter 2

Objectives

2.1 Autonomous Navigation

Objective: Develop a robot capable of autonomously navigating a modular field with various obstacles and uneven terrain.

2.2 Detailed Description:

2.2.1 Terrain Adaptability:

The robot must be designed to handle a wide range of terrains, including hills, uneven surfaces, and rubble. This requires robust mechanical design, with features such as high-torque motors, durable wheels, and a suspension system that can absorb shocks.

2.2.2 Obstacle Avoidance:

The field will contain numerous obstacles such as bricks, blocks, weights, and other large items. The robot must be equipped with sensors to detect and navigate around these obstacles without getting stuck. This includes real-time path planning and decision-making algorithms to choose the optimal route.

2.2.3 Line Following:

The primary navigation method involves following a black line on a white background. The robot must have high-precision sensors to detect this line and make quick adjustments to stay on course,

even when the line is interrupted by gaps or intersects with other lines.

2.2.4 Victim Identification

Objective: Ensure the robot can accurately identify and differentiate between live and dead victims using color and electrical conductivity.

2.3 Detailed Description:

2.3.1 Sensor Integration:

The robot will be equipped with sensors capable of detecting color and electrical conductivity. Live victims are represented by reflective silver spheres that are electrically conductive, while dead victims are black and non-conductive.

2.3.2 Algorithm Development:

Develop algorithms to process sensor data in real-time and accurately identify the type of victim. This involves using data from multiple sensors to reduce false positives and ensure reliable identification.

2.3.3 Environmental Adaptability:

The robot must be able to perform

Chapter 3

Hardware Components

3.1 Microcontroller

Component: Arduino

Description: The Arduino microcontroller serves as the central processing unit of the rescue robot, responsible for executing control algorithms, processing sensor data, and coordinating all system components. Its versatility and ease of programming make it an ideal choice for the development of autonomous systems.

3.2 Motors

Component: Two DC Motors for Movement

Description: The robot utilizes two DC motors to drive its movement across the rescue field. These motors provide the necessary torque and speed control to navigate various terrains, overcome obstacles, and transport victims. Their integration with the control system allows precise maneuverability and responsiveness to navigation commands.

3.3 Sensors

Component 1: MPU6050 Gyroscope and Accelerometer Sensor

Description: The MPU6050 sensor plays a crucial role in providing real-time data on the robot's orientation, acceleration, and rotational motion. This data is essential for maintaining balance,

stability, and precise navigation, especially when traversing uneven terrain or encountering obstacles. The integration of gyroscopic and accelerometer data enhances the robot's ability to adapt to dynamic environmental conditions.

Component 2: Infrared Sensors for Line Following

Description: Infrared sensors are employed to detect and follow the black line marking the designated path on the rescue field. These sensors enable the robot to maintain accurate alignment and course correction, ensuring smooth and efficient navigation along the predetermined route. By continuously monitoring the position of the line relative to the robot's sensors, it can make real-time adjustments to stay on track and avoid deviating from the desired path.

Component 3: LSA08 Line Tracking Sensor Array

Description: The LSA08 sensor array enhances the robot's line-following capabilities by providing a wider field of view and increased sensitivity to variations in the black line. This multi-sensor array improves the accuracy and reliability of line detection, enabling the robot to navigate complex intersections, sharp turns, and intricate patterns with greater precision. Its integration with the control system allows seamless integration into the robot's navigation algorithm, ensuring robust performance in challenging environments.

3.4 Actuators

Component: Motor Driver for Motor Control

Description: A motor driver is utilized to control the operation of the DC motors responsible for the robot's movement. This component serves as an interface between the microcontroller and the motors, regulating the voltage and current supplied to each motor to achieve desired speed, direction, and maneuverability. The motor driver ensures smooth and efficient motor operation while protecting the circuitry from overcurrent and voltage spikes, enhancing the overall reliability and performance of the robot's locomotion system.

3.5 Power Supply

Component: Battery Pack

Description: The robot is powered by a battery pack, providing the necessary electrical energy to operate all onboard components and systems. The battery pack offers portability, flexibility, and autonomy, allowing the robot to perform rescue missions in remote or inaccessible locations without reliance on external power sources. Its capacity and voltage output are carefully selected to meet the energy requirements of the robot's motors, sensors, microcontroller, and other electronic devices, ensuring sustained operation throughout the mission duration. Proper battery management and charging mechanisms are implemented to maximize battery life and optimize power efficiency, enabling prolonged and uninterrupted rescue operations.

Chapter 4

Software Components

4.0.1 Programming Language

Language: Arduino C/C++

Description: Arduino C/C++ serves as the primary programming language for developing the software components of the autonomous rescue robot. This language provides a robust and versatile framework for writing code that controls the behavior, functionality, and interaction of various system elements. Its simplicity and ease of use make it well-suited for rapid prototyping, debugging, and deployment in embedded systems like the rescue robot.

4.0.2 Libraries

Library 1: Wire.h for I2C Communication

Description: The Wire.h library facilitates Inter-Integrated Circuit (I2C) communication between the microcontroller and external devices, such as sensors and actuators. This communication protocol allows for bidirectional serial data transfer, enabling seamless integration and data exchange between multiple I2C-compatible devices connected to the same bus. By utilizing the Wire.h library, the rescue robot can interface with sensors, motor controllers, and other peripherals, enabling robust system control and sensor data acquisition.

```
#include <Wire.h>
```

Library 2: MPU6050_light.h for MPU6050 Sensor

Description: The MPU6050_light.h library provides a set of functions and utilities for interfacing with the MPU6050 sensor module, which combines a gyroscope and accelerometer into a

single integrated circuit. This library simplifies the process of initializing the sensor, reading raw sensor data, and performing basic calculations for orientation estimation and motion detection. By leveraging the capabilities of the MPU6050 sensor, the rescue robot can accurately measure its orientation, velocity, and acceleration, enabling precise navigation, stability control, and obstacle avoidance.

```
#include "MPU6050_light.h"
```

4.0.3 Snippets of Code with Explanation

Snippet 1: Initialization of I2C Communication

```
#include <Wire.h>

// Initialize I2C communication
Wire.begin();
```

Explanation: This code snippet initializes the I2C communication interface, allowing the microcontroller to communicate with external devices using the I2C protocol. The `Wire.begin()` function configures the microcontroller's I2C bus, enabling it to send and receive data from devices connected to the bus.

Snippet 2: Initialization of MPU6050 Sensor

```
#include "MPU6050_light.h"

MPU6050 mpu(Wire);

// Begin communication with MPU6050 sensor
byte status = mpu.begin();
```

Explanation: In this snippet, an instance of the MPU6050 sensor object is created, allowing the microcontroller to interface with the MPU6050 sensor module. The `mpu.begin()` function initiates communication with the sensor, establishing a connection and configuring its operational parameters. The `status` variable stores the status of the initialization process, enabling error checking and validation of sensor communication.

Chapter 5

System Design

5.1 Mechanical Design

Description: The mechanical design of the autonomous rescue robot is meticulously crafted to ensure robustness, agility, and adaptability to challenging terrains and obstacles commonly encountered in disaster scenarios.

- **Sturdy Chassis:** The robot is constructed on a sturdy chassis made of durable materials such as aluminum or reinforced plastic. This chassis provides a stable foundation for mounting components and withstands impacts and vibrations during operation.
- **Ample Ground Clearance:** The chassis design incorporates ample ground clearance to enable the robot to traverse over obstacles such as debris, rubble, and uneven terrain without getting stuck. This feature ensures continuous mobility and minimizes the risk of immobilization in hazardous environments.
- **Traction Wheels:** Specialized wheels with traction-enhancing treads are employed to provide optimal grip and traction on various surfaces, including concrete, asphalt, gravel, and dirt. These wheels facilitate smooth movement and effective navigation across different terrains, ensuring stability and control under diverse operating conditions.

5.2 Electrical Design

Description: The electrical design of the autonomous rescue robot focuses on the strategic placement of sensors and the integration of essential electronic components to enable accurate percep-

tion, control, and navigation capabilities.

- **Sensor Placement:** Sensors, including infrared sensors for line following and the MPU6050 sensor for orientation detection, are strategically positioned on the robot's chassis to optimize their field of view and sensitivity. This placement ensures comprehensive coverage of the surrounding environment, enabling efficient detection of the black line path and obstacles in the robot's path.
- **MPU6050 Sensor Integration:** The MPU6050 sensor module is seamlessly integrated into the electrical system to provide real-time orientation data, including pitch, roll, and yaw angles. This sensor's accurate measurement capabilities enable the robot to maintain balance, stability, and precise navigation while traversing uneven terrain and overcoming obstacles.

5.3 Software Design

Description: The software design of the autonomous rescue robot encompasses the implementation of sophisticated control algorithms, navigation logic, and victim identification strategies to enable autonomous operation and effective response in disaster scenarios.

- **PID Controller for Line Following:** The control logic incorporates a Proportional-Integral-Derivative (PID) controller to achieve smooth and accurate line following behavior. This controller dynamically adjusts motor speeds based on sensor feedback to keep the robot aligned with the black line path, ensuring precise navigation and maneuverability in complex environments.
- **Function-Specific Movement Commands:** Specific functions are designed and programmed to control the robot's movements in different directions, including forward, backward, left, right, and stop. These function-specific commands enable precise control and coordination of the robot's locomotion, facilitating efficient traversal of the rescue field and navigation around obstacles.
- **Victim Identification and Transportation Logic:** Advanced algorithms are implemented to enable the robot to identify and differentiate between live and dead victims using sensor data, such as color and electrical conductivity. Once identified, the robot

autonomously transports victims to designated evacuation points with care and precision, ensuring their safety and timely rescue during disaster scenarios.

By integrating these mechanical, electrical, and software design elements, the autonomous rescue robot achieves a comprehensive and effective system architecture capable of autonomously navigating treacherous terrain, identifying and transporting victims, and responding to dynamic challenges in disaster environments with efficiency and reliability.

Chapter 6

Software Architecture for Raspberry Pi-Based Autonomous Rescue Robot

6.1 Overview

The software architecture for our Raspberry Pi-based autonomous rescue robot is tailored to meet the specific requirements of navigating treacherous terrain, identifying victims, and executing rescue operations. It encompasses various layers and components designed to ensure seamless operation, robust performance, and efficient response to disaster scenarios.

6.2 Components of the Software Architecture

6.2.1 Operating System Layer

Raspbian OS

We utilize Raspbian OS as the operating system for the Raspberry Pi due to its compatibility with Raspberry Pi hardware and extensive support within the community.

6.2.2 Middleware Layer

GPIO Interface Library

This library provides access to the General Purpose Input/Output (GPIO) pins of the Raspberry Pi, allowing us to interface with sensors, actuators, and other hardware peripherals.

Sensor Drivers

Custom drivers are developed to interface with sensors such as infrared sensors for line following and the MPU6050 sensor for orientation detection.

Networking Libraries

Libraries for network communication protocols (e.g., MQTT, HTTP) enable data exchange between the robot and external systems.

6.2.3 Application Layer

Control Logic Module

This module implements the core control algorithms, including PID controllers for smooth line following and navigation.

Movement Control Module

Specific functions for controlling the robot's movements (forward, backward, left, right, stop) are implemented to facilitate precise locomotion.

Victim Identification Module

Algorithms for identifying and differentiating between live and dead victims based on sensor data (color and conductivity) are integrated into the software.

Victim Transportation Module

Logic for autonomously transporting victims to designated evacuation points is implemented, ensuring their safe evacuation from disaster zones.

6.2.4 Communication Layer

MQTT Messaging Protocol

MQTT is used for communication between the robot and external systems, such as remote servers or control centers. It enables efficient, lightweight messaging over TCP/IP, facilitating real-time data exchange and coordination.

HTTP REST API

HTTP-based RESTful APIs may be employed for interfacing with external services or platforms, allowing remote control, status monitoring, and data retrieval from the robot.

Custom Communication Protocols

Custom protocols may be developed for specialized communication requirements, such as inter-robot communication or integration with proprietary systems.

6.3 Key Considerations

6.3.1 Real-Time Responsiveness

The software architecture must ensure timely responses to sensor inputs and dynamic environmental conditions, especially during rescue operations where quick decision-making is crucial.

6.3.2 Fault Tolerance

Robust error handling mechanisms and fault tolerance strategies are implemented to handle unexpected failures or disruptions, ensuring the reliability and availability of the robot in challenging situations.

6.3.3 Modularity and Extensibility

The architecture is designed with modularity in mind, allowing individual components to be easily replaced, upgraded, or extended without affecting the overall system functionality.

6.4 Conclusion

The tailored software architecture for our Raspberry Pi-based autonomous rescue robot combines specialized control logic, sensor integration, and communication protocols to create a comprehensive and effective system for disaster response scenarios. By leveraging the capabilities of the Raspberry Pi platform and customizing the software stack to our specific requirements, we can develop a reliable and versatile autonomous robot capable of navigating hazardous environments and conducting successful rescue missions.

Chapter 7

Performance and Testing for Raspberry Pi-Based Autonomous Rescue Robot

7.1 Overview

The performance and testing of our Raspberry Pi-based autonomous rescue robot are essential aspects of ensuring its functionality, reliability, and effectiveness in real-world disaster scenarios. While we strive for optimal performance, it's crucial to maintain honesty and acknowledge potential limitations and challenges in our testing procedures and outcomes.

7.2 Performance Evaluation

7.2.1 Navigation Accuracy

Testing Approach

We assess the robot's ability to navigate accurately along predefined paths, including straight lines, curves, and intersections.

Honest Evaluation

While our robot demonstrates reasonable accuracy in following paths, occasional deviations may occur due to environmental factors such as uneven terrain or sensor inaccuracies.

7.2.2 Victim Identification and Transportation

Testing Approach

We evaluate the robot’s capability to differentiate between live and dead victims based on color and conductivity, as well as its effectiveness in safely transporting victims to evacuation points.

Honest Evaluation

Our robot reliably identifies victims and transports them to designated evacuation points in controlled test scenarios. However, real-world conditions may introduce complexities, such as varying lighting conditions or obstacles, which could impact performance.

7.2.3 Real-Time Responsiveness

Testing Approach

We assess the robot’s responsiveness to changing environmental conditions and dynamic obstacles encountered during navigation.

Honest Evaluation

While the robot demonstrates satisfactory responsiveness in most situations, occasional delays or hesitations may occur, particularly when encountering unexpected obstacles or complex terrain features.

7.3 Testing Procedures

7.3.1 Simulated Disaster Scenarios

Testing Approach

We simulate various disaster scenarios, including simulated rubble, debris, and uneven terrain, to evaluate the robot’s performance under realistic conditions.

Honest Evaluation

Our simulated scenarios provide valuable insights into the robot’s capabilities and limitations, but they may not fully replicate the complexity and unpredictability of real disaster environments.

7.3.2 Field Testing

Testing Approach

We conduct field tests in controlled outdoor environments to assess the robot’s performance under actual operating conditions.

Honest Evaluation

Field testing allows us to validate the robot’s performance in real-world settings, but factors such as weather conditions, terrain variations, and external interference may influence test results and introduce uncertainties.

7.4 Conclusion

While we strive for optimal performance in our Raspberry Pi-based autonomous rescue robot, it’s essential to maintain honesty and acknowledge the inherent challenges and limitations in our testing procedures and outcomes. By conducting thorough performance evaluations and testing in both simulated and real-world environments, we aim to continuously improve the robot’s capabilities and reliability, ensuring its effectiveness in assisting with disaster response efforts.

Chapter 8

Conclusion

In conclusion, our Raspberry Pi-based autonomous rescue robot shows promising performance in navigating treacherous terrain and conducting rescue missions. While there are occasional challenges, overall, the robot demonstrates reliability and effectiveness in controlled test scenarios.

Chapter 9

Future Improvements

Moving forward, we aim to enhance the robot's capabilities by focusing on improvements in sensor accuracy, real-time responsiveness, and robustness in handling unpredictable environments. Additionally, optimizing algorithms for victim identification and transportation will be a priority to further enhance the robot's efficiency in rescue operations.